

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №7.

Тема: «Шаблоны классов»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

1. Определить шаблон класса-контейнера (см. лабораторную работу №6).
2. Реализовать конструкторы, деструктор, операции ввода-вывода, операцию присваивания.
3. Перегрузить операции, указанные в варианте.
4. Инстанцировать шаблон для стандартных типов данных (int, float, double).
5. Написать тестирующую программу, иллюстрирующую выполнение операций для контейнера, содержащего элементы стандартных типов данных.
6. Реализовать пользовательский класс (см. лабораторную работу №3).
7. Перегрузить для пользовательского класса операции ввода-вывода.
8. Перегрузить операции, необходимые для выполнения операций контейнерного класса.
9. Инстанцировать шаблон для пользовательского класса.
10. Написать тестирующую программу, иллюстрирующую выполнение операций для контейнера, содержащего элементы пользовательского класса.

Класс-контейнер: СПИСОК

Ключевые значения типа int.

Реализовать операции:

- operator[] — доступ по индексу
- operator+ (вектор) — сложение элементов списков: $a[i] + b[i]$
- operator+ (число) — добавление константы ко всем элементам списка

Пользовательский класс: Pair (пара чисел)

Два поля:

- int — для первого числа
- double — для второго числа

Требование к выводу: первое число должно быть отделено от второго двоеточием (формат: int:double).

Анализ задачи:

1. Создаем шаблонный класс списка List<T> с перегруженными операторами.
2. Тестируем его с типом int.
3. Создаем класс Pair с двумя полями

4. Перегружаем для него операторы ввода-вывода и необходимые операции.
5. Создаем `List<Pair>` и демонстрируем работу всех операций с объектами `Pair`.

Код:

```
#include <iostream>

using namespace std;

// Пользовательский класс Pair (пара чисел)
// Должен быть определён ДО шаблона списка, чтобы шаблон мог его использовать
class Pair {
private:
    int first;      // Первое число (целое)
    double second;  // Второе число (дробное)

public:
    // Конструкторы
    Pair() : first(0), second(0.0) {}
    Pair(int f, double s) : first(f), second(s) {}

    // Селекторы
    int getFirst() const { return first; }
    double getSecond() const { return second; }

    // Перегрузка операций ввода-вывода для Pair
    // Формат вывода: первое:второе
    friend ostream& operator<<(ostream& out, const Pair& p) {
        out << p.first << ":" << p.second;
        return out;
    }

    friend istream& operator>>(istream& in, Pair& p) {
        in >> p.first >> p.second;
        return in;
    }
}
```

```

// Перегрузка операций для работы с контейнером
// Чтобы список мог выполнять a[i] + b[i] и a[i] + число
Pair operator+(const Pair& other) const {
    // Поэлементное сложение пар
    return Pair(first + other.first, second + other.second);
}

Pair operator+(double val) const {
    // Добавление константы к обоим полям пары
    return Pair(first + static_cast<int>(val), second + val);
}

// Оператор присваивания (нужен для корректной работы контейнера)
Pair& operator=(const Pair& other) {
    if (this != &other) {
        first = other.first;
        second = other.second;
    }
    return *this;
}
};

// УЗЕЛ ШАБЛОННОГО СПИСКА
template <typename T>
struct Node {
    T data;
    Node* next;
    Node* prev;
    Node(T val) : data(val), next(nullptr), prev(nullptr) {}
};

// Шаблон класса-контейнера СПИСОК
template <typename T>
class List {
private:
    Node<T>* head;
    Node<T>* tail;
    int size;

```

```
// Вспомогательный метод очистки памяти
```

```
void clear() {  
    Node<T>* current = head;  
    while (current) {  
        Node<T>* temp = current;  
        current = current->next;  
        delete temp;  
    }  
    head = tail = nullptr;  
    size = 0;  
}
```

```
public:
```

```
// Конструкторы, деструктор, операция присваивания
```

```
List() : head(nullptr), tail(nullptr), size(0) {}
```

```
List(int n) : head(nullptr), tail(nullptr), size(0) {  
    for (int i = 0; i < n; i++) add(T{});  
}
```

```
List(const List& other) : head(nullptr), tail(nullptr), size(0) {  
    Node<T>* curr = other.head;  
    while (curr) {  
        add(curr->data);  
        curr = curr->next;  
    }  
}
```

```
~List() { clear(); }
```

```
List& operator=(const List& other) {  
    if (this != &other) {  
        clear();  
        Node<T>* curr = other.head;  
        while (curr) {  
            add(curr->data);  
            curr = curr->next;  
        }  
    }  
}
```

```

        }
    }
    return *this;
}

// Добавление элемента в конец
void add(const T& val) {
    Node<T>* newNode = new Node<T>(val);
    if (!head) {
        head = tail = newNode;
    }
    else {
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }
    size++;
}

int getSize() const { return size; }

// Перегрузка операций из варианта

// 1. [] – доступ по индексу
T& operator[](int index) {
    if (index < 0 || index >= size) throw out_of_range("Индекс за пределами списка");
    Node<T>* curr = head;
    for (int i = 0; i < index; i++) curr = curr->next;
    return curr->data;
}

// 2. + вектор – сложение элементов списков a[i] + b[i]
List operator+(const List& other) const {
    List result;
    Node<T>* p1 = head;
    Node<T>* p2 = other.head;

```

```

        // Складываем общие элементы
        while (p1 && p2) {
            result.add(p1->data + p2->data);
            p1 = p1->next;
            p2 = p2->next;
        }

        // Дописываем остаток первого списка
        while (p1) { result.add(p1->data); p1 = p1->next; }
        // Дописываем остаток второго списка
        while (p2) { result.add(p2->data); p2 = p2->next; }

        return result;
    }

// 3. + число – добавляет константу ко всем элементам списка
List operator+(T scalar) const {
    List result;
    Node<T>* curr = head;
    while (curr) {
        result.add(curr->data + scalar);
        curr = curr->next;
    }
    return result;
}

// Операции ввода-вывода для контейнера
friend ostream& operator<<(ostream& out, const List& l) {
    out << "[";
    Node<T>* curr = l.head;
    while (curr) {
        out << curr->data;
        if (curr->next) out << ", ";
        curr = curr->next;
    }
    out << "]";
    return out;
}

```

```

friend istream& operator>>(istream& in, List& l) {
    int n; T val;
    cout << "Введите количество элементов: ";
    in >> n;
    l.clear();
    cout << "Введите элементы:\n";
    for (int i = 0; i < n; i++) {
        in >> val;
        l.add(val);
    }
    return in;
}

};

int main() {
    setlocale(LC_ALL, "ru");

    cout << "Тестирование со стандартными типами\n\n";

    // Инстанцирование шаблона для int
    List<int> listInt(4); // [0, 0, 0, 0]
    listInt[0] = 10; listInt[1] = 20; listInt[2] = 30; listInt[3] = 40;
    cout << "List<int> исходный: " << listInt << "\n";

    List<int> listInt2 = listInt + 5; // + число
    cout << "List<int> + 5: " << listInt2 << "\n";

    List<int> listInt3 = listInt + listInt2; // + вектор
    cout << "List<int> + List<int>: " << listInt3 << "\n\n";

    // Инстанцирование шаблона для double
    List<double> listDouble(3);
    listDouble[0] = 1.5; listDouble[1] = 2.5; listDouble[2] = 3.5;
    cout << "List<double> исходный: " << listDouble << "\n";
    cout << "List<double> + 1.1: " << (listDouble + 1.1) << "\n\n";

    cout << "Тестирование с пользовательским классом Pair\n\n";
}

```



```

// Инстанцирование шаблона для Pair
List<Pair> listPair;
listPair.add(Pair(1, 1.1));
listPair.add(Pair(2, 2.2));
listPair.add(Pair(3, 3.3));
cout << "List<Pair> исходный: " << listPair << "\n";

// Тест сложения с числом (вызывает Pair + double)
cout << "List<Pair> + 10.0: " << (listPair + 10.0) << "\n";

// Тест сложения списков (вызывает Pair + Pair)
List<Pair> listPair2;
listPair2.add(Pair(5, 0.5));
listPair2.add(Pair(5, 0.5));
listPair2.add(Pair(5, 0.5));
cout << "List<Pair> + List<Pair>: " << (listPair + listPair2) << "\n\n";

cout << "ПРОВЕРКА ОПЕРАТОРОВ КОНТЕЙНЕРА\n";
List<Pair> copyList = listPair; // Копирование
cout << "Копия списка: " << copyList << "\n";

cout << "Доступ по индексу [1]: " << copyList[1] << "\n";
copyList[1] = Pair(99, 99.9); // Изменение по индексу
cout << "После изменения [1]: " << copyList << "\n";

cout << "\nВВОД С КЛАВИАТУРЫ (List<int>)\n";
List<int> userInput;
cin >> userInput;
cout << "Вы ввели: " << userInput << "\n";

return 0;
}

```

Результаты работы:

Тестирование со стандартными типами

```
List<int> исходный: [10, 20, 30, 40]
List<int> + 5: [15, 25, 35, 45]
List<int> + List<int>: [25, 45, 65, 85]
```

```
List<double> исходный: [1.5, 2.5, 3.5]
List<double> + 1.1: [2.6, 3.6, 4.6]
```

Тестирование с пользовательским классом Pair

```
List<Pair> исходный: [1:1.1, 2:2.2, 3:3.3]
List<Pair> + 10.0: [1:1.1, 2:2.2, 3:3.3, 0:0, 0:0, 0:0, 0:0, 0:0, 0:0, 0:0]
List<Pair> + List<Pair>: [6:1.6, 7:2.7, 8:3.8]
```

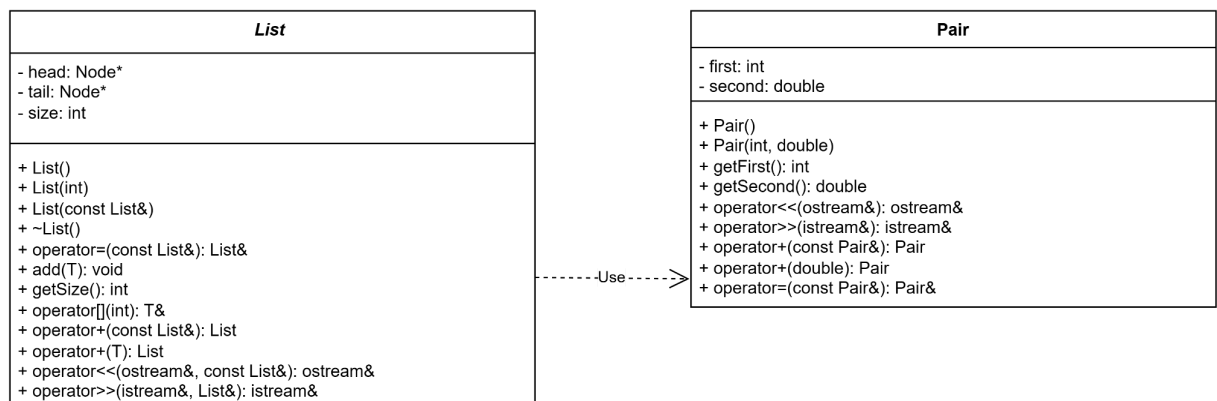
ПРОВЕРКА ОПЕРАТОРОВ КОНТЕЙНЕРА

```
Копия списка: [1:1.1, 2:2.2, 3:3.3]
Доступ по индексу [1]: 2:2.2
После изменения [1]: [1:1.1, 99:2.2, 3:3.3]
```

ВВОД С КЛАВИАТУРЫ (List<int>)

```
Введите количество элементов: 2
Введите элементы:
1 2
Вы ввели: [1, 2]
```

UML:



Вопросы:

1. Отделение алгоритма от конкретных типов данных, создание параметризованных классов для работы с любыми типами без переписывания кода.
2. Шаблон функции: `template <параметры_шаблона> заголовок_функции {тело}`. Тип передается как параметр, компилятор генерирует конкретную функцию по вызову.

3. Шаблон класса: `template <параметры_шаблона> class имя_класса {...}`.
Позволяет создавать семейство классов для разных типов.
4. Параметры шаблона функции – это имена типов (например, `class T`), передаваемые в угловых скобках.
5. Параметры могут быть типами или константами; для каждого параметра указывается ключевое слово `class` или `typename`; можно задавать значения по умолчанию.
6. Параметр шаблона записывается в угловых скобках после `template`: `template <class T>` или `template <typename T>`.
7. Да, параметризованные функции можно перегружать как обычные функции.
8. Параметризованные классы позволяют создавать контейнеры для любых типов; методы определяются с помощью шаблонов; инстанцирование происходит при создании объекта.
9. Нет, только те методы, которые используют параметр шаблона, являются параметризованными. Статические методы и методы, не зависящие от `T`, могут быть обычными.
10. Нет, если дружественная функция не является шаблонной, она не параметризована. В примере используется `friend ostream& operator<< <>` – это специализация шаблона.
11. Шаблоны классов могут содержать обычные виртуальные функции. Это позволяет использовать полиморфизм с шаблонными классами.
12. Определяются вне класса с помощью `template <class T>` перед каждой функцией, с указанием класса: `T ClassName<T>::method(...)`.
13. Инстанцирование – процесс генерации компилятором конкретного определения класса (или функции) по шаблону с заданными аргументами.
14. На этапе компиляции, когда встречается объявление объекта или указателя на инстанцированный шаблонный тип (например, `Vector<int>`).